Concurrent Programming: Languages and techniques

Rust Concurrency

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Computer Science,
NOVA School of Science and Technology

Concurrency on Rust – safely relaxing linearity

Key goal: no races

Guarantee Mutual Exclusive Access to References

`Mutex`: **Threads waiting for the lock are not available**

- Prevents multiple readers,
  even when it is *safe* – no writers
- Prevents readers when there is a writer

The problem: AXM (alias XOR mutation) discipline

Data sharing policies

- Default: references have a single owner –
  can read, write, and deallocate
- Shared: all aliases can read; no one can write
- Mutable: one alias can read and write; no one else have access

Implementation of data-strutures

- Easy for acyclic (tree-like) – at most one pointer to each node
- Hard for cyclic (graph-like) – at least one pointer to some node

But never doubly linked!

How Rust deals with mutation: 1. To declare variables

By default, variables are immutable (like "constants")

```
let x:i32 = 42; // an integer constant
x *= 2 // Error: cannot assign twice to immutable variable
let y:&i32 = &x; // an immutable reference
y = &x; // Error: cannot assign twice to immutable variable
```

Mutable variables need to be explicitly declared

```
let mut z = x; // an integer variable (type implicit)
z *= 2;
let mut w = &x; // a mutable reference
w = &z
```

How Rust deals with mutation: 2. To declare (mutable) borrows

By default, references are uniquely owned.

Borrow modes

- Immutable: `let` y = &x // x and y are read-only
- Mutable: `let` y = &`mut` x // x unusable; y read/write

(im)mutable borrows

```
let mut x = [1, 2, 3]; // a static array
let y = &x[0]; // immutable borrow
myfun(&mut x); // myfun is a function with a mutable borrow
Error: immutable borrow used
```

Unsafe code:

reference y might hold a dangling pointer

Mutable borrows: more examples*

Consider the code snippet:

```
let mut x = ...; // x mutable and owner (read/write access)
let a = &mut x; // x unusable; a read, *a write
let b = &(*a); // *a read; *b, b read (*a, *b refer the borrow)
```

Consider now:

```
fn myfun(arr: &mut [i32;3]){...} // mutable borrow returned
let mut x = [1,2,3];
myfun(&mut x);
let y = &mut x // x may be borrowed mutably once more
```

* "Short Paper: Rusty Types for Solid Safety". Sergio Benitez. DOI 10.1145/2993600.2993604

How Rust deals with mutation: 3. To declare mutable *types*

Syntax: `&mut` T

type of mutable, hence *linear* (not shareable) references

Example: a function with a parameter with a mutable reference type

```
fn myfun(arr: &mut [i32;3]){...}
fn mut_vec(v: &mut Vec<i32>) {...}
```

A binary tree in Rust

```
struct Node<T> {
    data:T,
    left:Option<NodeRf<T>>,
    right:Option<NodeRf<T>>
}
```

Why `Option`: node can be a leaf

```
type NodeRf<T> = Box<Node<T>>
```

`Box`

the type of a pointer who owns what it points to

A binary tree in Rust

With a shared reference to the root of the tree

one may read the values in all nodes

`Box` **is mutable**

With a mutable reference to the root of the tree

one may change the values in any node

How to use a concurrent tree-like data structure?

protect the nodes to avoid races:

`type NodeRf<T> = &Mutex<Node<T>>`

Concurrency severely restricted: what if there are only readers?

RWLock: at any point in time,

allows any number of readers XOR at most one writer

```rust
use std::sync::RwLock;
let lock = RwLock::new(5);
{ // many reader locks can be held at once
  let r1 = lock.read().unwrap();
  let r2 = lock.read().unwrap();
  assert_eq!(*r1, 5);
  assert_eq!(*r2, 5);
} // read locks are dropped at this point
{ // only one write lock may be held
  let mut w = lock.write().unwrap();
  *w += 1;
  assert_eq!(*w, 6);
} // write lock is dropped here
```

RWLock: sharing via threads requires Arc

```
let count = Arc::new(RwLock::new(5));
let c1 = Arc::clone(&count);
let h1 = thread::spawn(move ||
{
  let r1 = c1.read().unwrap();
  println!("{}",*r1);
});
let c2 = Arc::clone(&count);
let h2 = thread::spawn(move ||
{
  let r1 = c2.read().unwrap();
  println!("{}",*r1);
});
```

Condition Variables

- block threads to avoid consuming CPU time while waiting for an event to occur
- are associated with a boolean predicate (a condition) and a mutex
- the predicate is verified inside of the mutex to determine if a thread must block

Inside a lock, spawn a new thread and wait for it to start

```
use std::sync::{Arc,Mutex,Condvar};
use std::thread;
let pair = Arc::new((Mutex::new(false),Condvar::new
let pair2 = Arc::clone(&pair);
thread::spawn(move|| {
let (lock, cvar) = &*pair2;
  let mut started = lock.lock().unwrap();
  *started = true;
  cvar.notify_one(); // Notify cvar the value change
}); // Wait for the thread to start up
let (lock, cvar) = &*pair;
let mut started = lock.lock().unwrap();
while !*started {
  started = cvar.wait(started).unwrap();
}
```

Beware of deadlocks

```
thread::spawn(move|| { // spawn new thread
  let (lock, cvar) = &*pair2;
  let mut started = lock.lock().unwrap();
  *started = true;
  cvar.notify_one(); // notify the value changed
}); // wait for the thread to start up
let (lock, cvar) = &*pair;
let mut started = lock.lock().unwrap();
while !*started {
  started = cvar.wait(started).unwrap(); }
```

- `wait` atomically *releases* the lock and blocks the thread
- Thread resumes when `notify_one` or `notify_all` are called
  – lock reacquired!
- one can also wait as long as some cond., or until timeout

`wait_while` locks the current thread until cond. var. receives notification and the provided condition is false

```
use std::sync::{Arc,Mutex,Condvar};
use std::thread;
let pair = Arc::new((Mutex::new(true),Condvar::new(
let pair2 = Arc::clone(&pair);
thread::spawn(move || {
  let (lock, cvar) = &*pair2;
  let mut pending = lock.lock().unwrap();
  *pending = false;
  cvar.notify_one();
});
let (lock, cvar) = &*pair;
// wait while value inside `Mutex<bool>` is `true`
let _guard = cvar.wait_while(lock.lock().unwrap(),
  |pending| { *pending }).unwrap();
```

Asynchronous Concurrency

Rust threads are 1:1 to OS threads

- CPU and memory demanding

How to run a (large) number of concurrent tasks on a small number of OS threads?

- run several concurrent tasks on a single thread

Async Runtimes

- Thread spawning, IO, and specific instructions rely on dedicated runtime environments
- Tokio is a popular one
- Manage *task systems* – lightweight threading

Asynchronous Concurrency - main instructions

Async

launches a block of code to be executed in parallel with the rest of the code

Similar to spawn

Await

makes a block of code stand by until another completes execution

Similar to lock, but asynchronous (does not "stop the world")

Futures and `block_on`:

```rust
use futures::executor::block_on;
async fn hello_world() {
  println!("hello, world!");
}
fn main() {
  let future = hello_world(); // Nothing is printed
  block_on(future); // `future` runs and prints
                    // "hello, world!"
}
```

`block_on`:

stops the current thread until the *future* has run to completion

Futures are *lazy*

do nothing until they are run

Futures: *eventual* values produced by asynchr. computations

```
let pool = ThreadPool::new().expect("Failed");
let (tx, rx) = mpsc::unbounded::<i32>();
let fut_values = async {
  let fut_tx_result = async move {
    (0..100).for_each(|v| {
      tx.unbounded_send(v).expect("Failed");
    })
  };
  pool.spawn_ok(fut_tx_result);
  let fut_values = rx.map(|v| v * 2).collect();
  fut_values.await
};
let values: Vec<i32> = executor::block_on(fut_values
println!("Values={:?}", values);
```

Futures and `block_on`

```
async fn learn_song() -> Song { /* ... */ }
async fn sing_song(song: Song) { /* ... */ }
async fn dance() { /* ... */ }

fn main() {
  let song = block_on(learn_song());
  block_on(sing_song(song));
  block_on(dance());
}
```

Instead of blocking a thread, how to make it wait asynchronously for the future to complete?

Waiting asynchronously in multiple futures

```
async fn learn_and_sing() {
// Wait until the song has been learned to sing it.
// `.await` allows to `dance` at the same time.
  let song = learn_song().await;
  sing_song(song).await;
}
async fn async_main() {
  let f1 = learn_and_sing();
  let f2 = dance();
  // `join!` like `.await` but
  // waits for multiple futures concurrently.
  futures::join!(f1, f2);
}
```

Waiting asynchronously in multiple futures

- If temporarily blocked in the ‘`learn_and_sing`‘ future, the ‘`dance`‘ future will take over the current thread
- If ‘`dance`‘ blocked, ‘`learn_and_sing`‘ can take back over
- If both futures are blocked, then ‘`async_main`‘ is blocked and will yield to the executor.

```
// Wait until the song has been learned to sing it.
// ‘.await‘ allows to ‘dance‘ at the same time.
async fn learn_and_sing() { ... }
async fn async_main() {
  let f1 = learn_and_sing();
  let f2 = dance();
  // ‘join!‘ waits for multiple futures concurrently
  futures::join!(f1, f2);
}
```

Two ways to use async

```
async fn foo() -> u8 { 5 }
fn bar() -> impl Future<Output = u8> {
  async {
    let x: u8 = foo().await;
    x + 5
  }
}
```

- ‘`foo()`‘ returns a type that implements ‘`Future<Output = u8>`‘
  ‘`foo().await`‘ will result in a value of type ‘`u8`‘
- ‘`async`‘ block results in a type that implements ‘`Future<Output = u8>`‘

Ways to use futures

1. One at a time with `.await`
2. Running to completion, with `spawn` (a top-level task)
3. Waiting on a set to complete, with `join!`
4. Waiting on one in a set to complete, with `select!`
5. Waiting on a set of copies of the same future, with `FuturesUnordered`

Async and Ownership

async functions which take references return a Future bounded by the lifetime of the arguments

```
// This function:
async fn foo(x: &u8) -> u8 { *x }
// Is equivalent to this function:
fn foo_expanded<'a>(x: &'a u8) ->
  impl Future<Output = u8> + 'a {
  async move { *x }
}
```

Async and Ownership

Lifetime of borrowed reference must match the future's one

```
fn bad() -> impl Future<Output = u8> {
  let x = 5;
  borrow_x(&x) // ERROR: `x` does not live long enou
}

fn good() -> impl Future<Output = u8> {
  async {
    let x = 5;
    borrow_x(&x).await
  }
}
```

Async and Move

```
async fn blocks() {
  let my_string = "foo".to_string();
  let future_one = async {// ...
    println!("{my_string}");  };
  let future_two = async {// ...
    println!("{my_string}");};};
//Run both futures to completion; prints "foo" twice
let (()),()) = futures::join!(future_one,future_two);
}

fn move_block() -> impl Future<Output = ()> {
  let my_string = "foo".to_string();
  async move {// ...
    println!("{my_string}");
  }
}
```

Tokio asynchronous runtime

Multi-threaded asynchronous version of the standard library
with a large ecosystem of libraries

```
async fn say_world() {
  println!("world");
}

#[tokio::main]
async fn main() {
  // Calling `say_world()` does not execute.
  let op = say_world();
  // This println! comes first
  println!("hello");
  // Calling `.await` on `op` executes `say_world`.
  op.await;
}
```

Tokio asynchronous runtime

Concurrency requires spawning new tasks

```
use tokio::net::TcpListener;

#[tokio::main]
async fn main() {
  let lstnr =
    TcpListener::bind("127.0.0.1:6379").await.unwrap
  loop {
    let (socket,_) = lstnr.accept().await.unwrap();
    // new task spawned for each inbound socket.
    // socket moved to new task and processed there
    tokio::spawn(async move {
      process(socket).await;
    });
  }
```

Tokio asynchronous runtime

Concurrency requires handling running tasks

```rust
#[tokio::main]
async fn main() {
  let handle = tokio::spawn(async {
    // Do some async work
    "return value"
  });
  // Do some other work
  let out = handle.await.unwrap();
  println!("GOT {}", out);
}
```